

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

Multi-robot persistent surveillance with connectivity constraints

JÜRGEN SCHERER, BERNHARD RINNER

Institute of Networked and Embedded Systems, University of Klagenfurt, Austria

Corresponding author: Jürgen Scherer (e-mail: juergen.scherer@aau.at).

ABSTRACT Mobile robots, especially unmanned aerial vehicles (UAVs), are of increasing interest for surveillance and disaster response scenarios. We consider the problem of multi-robot persistent surveillance with connectivity constraints where robots have to visit sensing locations periodically and maintain a multi-hop connection to a base station. We formally define several problem instances closely related to multi-robot persistent surveillance with connectivity constraints, i.e. connectivity-constrained multi-robot persistent surveillance (CMPS), connectivity-constrained multi-robot reachability (CMR), and connectivity-constrained multi-robot reachability with relay dropping (CMRD), and show that they are all NP-hard on general graphs. We introduce three heuristics with different planning horizons for convex grid graphs and combine these with a tree traversal approach, which can be applied to a partitioning of non-convex grid graphs (CMPS with tree traversal, CMPSTT). In simulation studies we show that a short horizon greedy approach, which requires parameters to be optimized beforehand, can outperform a full horizon approach, which requires a tour through all sensing locations, if the number of robots is larger than the minimum number of robots required to reach all sensing locations. The minimum number required is the number of robots necessary for building a relay chain to the farthest sensing location from the base station. Furthermore, we show that partitioning the area and applying the tree traversal approach can achieve a similar performance to the unpartitioned case up to a certain number of robots but requires less optimization time.

INDEX TERMS Mobile robots, Multirobot systems, Path planning, Persistent surveillance, Complexity analysis

I. INTRODUCTION

MOBILE robots, especially unmanned aerial vehicles (UAVs), are of increasing interest in various application domains including surveillance. Examples for the deployment of robots for surveillance scenarios are disaster response [1], [2], [3], wildfire monitoring [4], security tasks [5], environmental monitoring [6], and exploration and mapping [7], [8]. Persistent surveillance is the task of continuously monitoring an environment over a long period of time. The potentially large area and the limited sensor view of robots requires a movement strategy such that every point of interest in the area gets visited periodically by the robots. In disaster response scenarios, it is also crucial that the mission operators are aware of the situation at any time during the mission. This implies that the robots have to continuously report the state of the mission and the captured data to the base station. Wireless transceivers enable the robots to exchange data over a limited distance only, and therefore it is necessary to transmit the data over multiple hops if a continuous connectivity

to the base station is required and the area is larger than the communication range. A continuously connected network of robots and the base station further allows to track the state of the (aerial) robots for safety reasons.

In this paper we investigate the problem of persistent surveillance with continuous connectivity constraints. Given a representation of the area, the number of robots, the positions of the points of interest (which we denote sensing locations) and the base station, the problem is to find a path for each robot that minimizes the worst idleness for all sensing locations such that the network of robots and base station is connected throughout the mission. The idleness of a sensing location at a certain instant is defined as time that has passed since the last visit by any robot, and the worst idleness is the maximum idleness over all sensing locations and over the whole mission duration. Since the duration of a persistent surveillance mission is potentially infinite, strategies are necessary that generate solutions for an infinite time horizon. Due to the connectivity constraint,

the robots mutually restrict their possible movements and traditional patrolling strategies (e.g. [9], [10], [11]) cannot be directly applied because they do not coordinate the robots' movement in space and time for maintaining continuous connectivity. Online multi-robot persistent surveillance algorithms typically require the state of the whole environment to make a decision about the next action. Therefore, they rely implicitly on some communication mechanism allowing the robots to exchange information but do not consider a limited communication range [12], [13].

Path planning for mobile robots is often based on an abstract representation of the environment obtained by some discretization technique. In this work we consider graphs [14] and grids [12]. In a graph representation two types of edges describe whether a robot can move between two vertices and whether robots can communicate when placed at two different vertices at the same time. A grid represents a special type of graph where a robot can move between the neighboring cells. We study different problem instances related to connectivity-constrained multi-robot persistent surveillance. In particular, we define the problems connectivity-constrained multi-robot persistent surveillance (CMPS), connectivity-constrained multi-robot reachability (CMR), and connectivity-constrained multi-robot reachability with relay dropping (CMRD). The latter two problems are concerned with reachability of vertices when all robots start at a dedicated base station vertex. We show that all these problems are NP-hard on graphs.

Figure 1 illustrates the different problems in a grid-based scenario composed of four convex partition around an obstacle. Sensing robots aim to cover the partitions while maintaining connectivity to the base station with the help of relay and release robots. The relay and release robots do not move while a partition is covered. Release robots are robots that stay at a release point. The release point is the starting position where the sensing robots start to cover the partition after they have moved together with the release robot to the release point. CMPS considers the movement planning within one convex area, CMR deals with the question which vertices/cells are reachable from the base station, and CMRD deals with the problem of placing relay robots. CMPSTT is concerned about the order of covering the partitions and the number of robots assigned to partitions. We assume that robots can change roles and can take on every role.

Due to the complexity of finding feasible solutions, we do not attempt to solve CMPS on general graphs. In previous works [15], [16] we investigated different strategies on grids where cells within a certain distance are within communication range. In contrast to our previous work we do not consider energy constraints in this work. These strategies can only be applied on convex grids without holes, which arise in the presence of obstacles. A tree traversal algorithm [17] with relay dropping can be applied to more general environments but does not perform as well as the suggested strategies in convex scenarios with many sensing locations. Therefore, we suggest a combination of tree traversal and coverage (CMPS

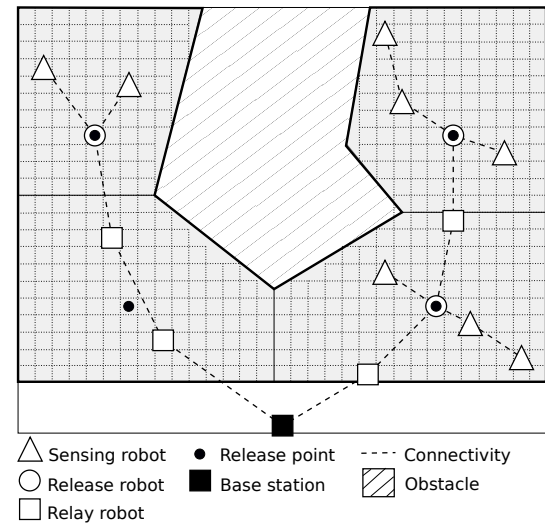


FIGURE 1. Problem instances of multi-robot persistent surveillance with connectivity constraints depicted on a grid environment with four convex partitions around an obstacle. The lower left partition is currently not covered in the depicted scenario. In a potential solution for CMPSTT the robots in the upper left partition gather at the release robot and retreat together with the relay robot to the release point of the lower left partition to cover this partition.

with tree traversal, CMPSTT), which can be applied after a partitioning of an arbitrary shaped environment into convex partitions and ensures the connectivity constraint. We adapt the algorithm of [17] and show that determining the optimal order for visiting the partitions of a given partitioning to minimize the worst idleness is also NP-hard. Note that we have studied related patrolling problems of minimizing or constraining the delay between data generation at the robots and data arrival at the base station with *relaxed* connectivity constraints in [18] and [19].

The contributions of this work can be summarized as follows: (i) We define CMPS, CMR, CMRD on general graphs and prove that they are NP-hard, (ii) we propose movement strategies for convex grid areas, (iii) we combine tree traversal and area coverage for partitioned environments and show that CMPSTT is NP-hard, and (iv) we perform an extensive simulation study to assess the performance of the proposed strategies.

The remainder of the article is organized as follows: Section II discusses related work, Section III introduces the notation and investigates the complexity on general graphs, Section IV describes the strategies on convex grid areas, Section V describes an extension to partitioned areas, Section VI presents the results of the simulation studies, and Section VII concludes the paper.

II. RELATED WORK

Multi-robot persistent surveillance is related to the multi-robot patrolling problem. This problem is usually concerned with determining closed paths, which are continuously traversed by robots, or with controlling the robots along predefined paths to optimize for some performance metric [20],

[21], [22], [10]. Tour planning is often tackled from the operations research perspective by solving a vehicle routing problem or a multi-traveling salesperson problem on a graph. In [23] tours for multiple agents are planned such that the visit frequency of targets is maximized and agents can refuel at refuel depots. Mersheeva and Friedrich [24] perform path planning for multiple UAVs that repeatedly visit targets with different priorities. UAVs change batteries at base stations, and the planning horizon is determined by the available number of batteries. Manyam et al. [25] consider also the data delivery latency to the base station. Keller et al. [26] present a tour planning approach taking into account turning radius constraints of gliders. In [27] trajectories are planned to estimate a spatio-temporal field in a dynamic environment. A key difference of these tour planning approaches to our work is that continuous connectivity of the robots to a base station has not been considered at all.

Multi-robot motion control is another related topic for persistent surveillance. Nigam et al. [12] introduce a control strategy where each UAV selects the next cell to visit based on the idleness of the cell, the distance between the UAV and the cell, and the distances between the other UAVs and the cell. The infinite horizon persistence surveillance problem is converted to a short horizon problem where the necessary parameters are determined by an offline optimization approach. Franco et al. [13] present a controller adjusting speed and direction to reach a desired long term coverage profile. Santos et al. [8] investigate exploring a spatio-temporal field with different goal selection strategies for a single robot. If the motion planning is done online with the approaches in [12], [13], the robots need to know the state of the whole environment and of all other robots. This knowledge requires communication between the robots or the base station and the robots. However, the discussed related work does not consider limited communication range.

There exist many variations of the multi-robot connectivity problem in literature. Connectivity preserving control-theoretic approaches using local coordination have been proposed for applications like rendezvous [28] and reconfiguration while maintaining a given connectivity graph (e.g. a tree) [29]. Sabattini et al. [30] describe how estimation of the global connectivity (with global coordination) allows to maintain connectivity without maintaining a predefined graph. Zavlanos et al. [31] present a distributed hybrid controller, which reaches consensus on a common tree that the robots preserve while leader robots pursue goal locations. Khateri et al. [32] discuss and compare local and global coordination strategies. They conclude that global methods result in better performance for coverage missions in the presence of robot failures (immobilized robots) but might be inferior to local methods for large robot teams. The reason is that large communication delays might force robots to slow down their movement. Some approaches extend the simple on-off connectivity model by incorporating network parameters to ensure network integrity [33] or to meet bandwidth requirements of certain tasks [34], [35].

Navaravong et al. [36] provide a brief survey on formation and connectivity control of mobile robot networks and conclude that these two problems can be decomposed such that connectivity control can happen at discrete space over network graphs, whereas motion can be controlled with navigation functions. Different strategies for particular applications and connectivity requirements have been proposed in the literature that separate (possibly centralized) computation of a task assignment or a final configuration from (distributed) connectivity preserving navigation [37], [38] [39].

Precomputing paths considering connectivity constraints requires prior knowledge (e.g. in the form of communication maps) whether two robots at different locations will be able to communicate with each other. In contrast to online algorithms that are able to estimate the link quality and prevent robots from disconnecting the network (control-based approaches, e.g. [33]), the goal of offline path planning approaches is to precompute paths such that robots reach a final configuration (e.g. [40], [41], [42]). Two ways to generate communication maps describing the connectivity in an environment are exploration [43], and simulation [44]. Exploration requires at least two robots that move around in the environment according some strategy and record the link quality between different locations. Network simulators use a map of the environment and signal propagation models to estimate the link quality between different locations, and this information can be used for path planning.

Relaxing the continuous connectivity constraint allows the robots to disconnect and meet (e.g. at periodic intervals) for coordination of the mission execution or for exchanging collected data with other robots or a base station. Intermittent connectivity is a requirement in various applications and requires determining when and where which robots should meet. There are also various multi-robot applications with intermittent connectivity including multi-robot exploration [40], [45], [46], multi-UAV inspection and surveillance [47], [48], [49], search-and-rescue [50], multi-UAV persistent surveillance [20], [25].

Maintaining connectivity while executing spatially distributed tasks is a recurrent requirement in mobile-robot systems. Panerati et al. [37] present an algorithm for a swarm of robots that moves in compact formation through a convex environment and stops at certain locations to visit multiple task locations simultaneously by building one chain of robots towards each task location. The order in which the tasks should be visited is determined by solving a mixed integer linear program (MILP). A tree growing algorithm presented by Majcherczyk et al. [39] arranges the robots into a star-like topology to visit multiple task locations. Ponda et al. [51] present an algorithm for allocation of data streaming and relaying tasks to maintain connectivity to a base station during task execution. Grøtli and Johansen [34] investigate offline mission planning for multiple UAVs that visit targets and stream data to a base station by solving a MILP model and exploiting a radio propagation path loss simulator to take bandwidth requirements into account. Flushing et al.

[48] present a MILP model for task scheduling as well as data routing and transmission scheduling. Zavlanos et al. [33] present a distributed algorithm for maintaining communication requirements with a fixed infrastructure. All these approaches focus on connectivity maintenance with or without explicit task allocation but do not explicitly consider a persistent visitation of a given set of tasks. Solving MILPs for path planning problems is demanding due to the computational complexity of integer programming.

The computational complexity of planning the robots' movement on graphs has been investigated in literature. Hollinger and Singh [40] show that the problem of planning paths for multiple robots from start to goal positions that maintain connectivity in every time step is NP-hard on graphs, but some details about how to construct the graph are missing. Tateo et al. [41] investigate a related problem, which they call multi-agent connected path planning problem (MCP), and show that it is PSPACE-complete. Anisi et al. [52] consider the related problem of planning a path in a visibility graph such that several targets can be observed and show that it is NP-hard. Banfi et al. [14] show that the problem of planning paths that reach a connected configuration on a graph in minimum time is NP-hard. In contrast to all these investigations we focus on persistent surveillance with the connectivity requirement to a base station.

As a summary, our work combines persistent surveillance with connectivity requirements and appropriate motion planning. While there exists a lot of work on each of these problems, we are not aware of related work that can be directly applied to our problem. The considered exploration task in [17] represents the most relevant work on connectivity-constrained motion planning with goal selection, and we use it as a baseline for our simulation study. We employ the work in [12] on goal selection for persistent surveillance, and a comparison with other approaches for persistent surveillance has been provided in [12]. Work that approach connectivity-constrained motion from a control-theoretic perspective (e.g. [33], [31], [29], [39]) do often not consider a particular application and goal selection. As a consequence, goal selection and scheduling of the robots for persistent surveillance has to be incorporated into these approaches.

III. PROBLEM FORMULATION AND COMPLEXITY ANALYSIS

This section introduces the notation and provides formal definitions of the problems. The used symbols and abbreviations are summarized in Table 1. A set $R = \{1, \dots, r\}$ of robots is available for the surveillance of an environment, which is modeled with two graphs: the movement graph $G_M = (V, E_M)$ and the connectivity graph $G_C = (V, E_C)$. Both graphs share the same set of vertices V (with $|V| = n$), which describe possible positions of robots at discrete points in time. Time is divided into time steps, and the positions of the robots at time step t is denoted $p_t = (p_t(1), \dots, p_t(r))$, with $p_t(i) \in V$. A subset $V_S \subseteq V$ represents sensing locations. If there is an edge $[v, w] \in E_M$, a robot can move

Symbol	Meaning
$G_M = (V, E_M)$	(undirected) movement graph with vertex set V and edge set E_M
$G_C = (V, E_C)$	(undirected) connectivity graph with vertex set V and edge set E_C
$G_C(W)$	vertex induced subgraph of G_C (defined by vertex set W)
b	base station vertex, or base station partition
$[v, w] \in E$	(undirected) edge between v and w
V_S	set of sensing location vertices
n	number of vertices in V
$R = \{1, \dots, r\}$	set of r robots
$p_t(i)$	position of robot i at time t
p_t	vector of positions of all robots at time t
π	sequence of all robot positions
G_C^t	vertex induced subgraph of G_C at time t (defined by set of vertices in p_t and b)
$I_t^\pi(v)$	instantaneous idleness of vertex v at time t (using π)
$WI_t^\pi(v)$	instantaneous worst idleness of vertex v at time t (using π)
WI	worst idleness
CT	coverage time
$dist_G(s, d)$	length of shortest path between vertices s and d in graph G
$\mathbb{R}_{\geq 0}, \mathbb{R}_{> 0}$	set of real numbers greater than or equal 0, and greater than 0, respectively
CMPS	connectivity-constrained multi-robot persistent surveillance
CMPSTT	CMPS with tree traversal
CMR	connectivity-constrained multi-robot reachability
CMRD	CMR with relay dropping
FH	full horizon algorithm
SH	short horizon algorithm
SHC	short horizon cooperative algorithm
TT	tree traversal algorithm

TABLE 1. List of symbols and abbreviations.

from v to w within one time step. An edge $[v, w] \in E_C$ means that two robots, or a robot and the base station $b \in V$, are able to transfer data to each other when they are at positions v and w at the same time. We call the tuple (G_M, G_C, b, V_S) describing an environment simply “graph” and refer explicitly to the graphs G_M and G_C if necessary.

A patrolling solution π is a mapping from instants of time to vertices in V for every robot and describes when vertices are visited by the robots. At each time step an instantaneous idleness is associated with a vertex $v \in V_S$. This value describes the time that has passed since the last visit of the vertex. The definition of the idleness criterion adheres to the definition in [11]:

Definition 1 (Instantaneous idleness, instantaneous worst idleness, worst idleness criterion [11]). *If the robots follow a solution π , the instantaneous idleness $I_t^\pi(v) \in \mathbb{R}_{\geq 0}$ at time t of vertex $v \in V_S$ is the elapsed duration since the last visit of v by any robot. By convention, the instantaneous idleness at $t = 0$ is zero for any solution π and each $v \in V_S$ ($I_0^\pi(v) = 0$). The worst idleness criterion WI^π is defined as*

$$WI^\pi := \limsup_{t \rightarrow +\infty} WI_t^\pi \quad (1)$$

where WI_t^π is the instantaneous worst idleness and is defined as $WI_t^\pi := \max_{v \in V_S} I_t^\pi(v)$.

A solution of a persistent surveillance problem can be described by a sequence of positions $\pi := (p_0, p_1, \dots)$ where each p_{t+1} results from p_t when the robots move along the edges of E_M (or stay at the same vertex). A valid solution for CMPS has the property that at each time step t the vertex induced subgraph $G_C^t := G_C(\{p_t(i) : i \in R\} \cup \{b\})$ is connected, i.e. all robots are connected with the base station. Note that multiple robots can move to the same vertex at the same time.

To analyze the complexity of the problem of finding a minimum WI solution, we define CMPS as decision problem (the problem of deciding whether a given tuple belongs to a set of tuples):

Definition 2 (d-CMPS). Problem *d-CMPS* is described by a set of tuples of the form $(G_M, G_C, b, V_S, p_0, 1^T)$ where $p_0 \in V^r$ are the initial positions of the robots and T is a time bound. The elements of the set have the properties: (i) $p_0(i) = v \in V_S$ for some robot i , (ii) there is a sequence $(p_0, \dots, p_{t'})$, $t' \leq T$ such that $p_{t'} = p_0$, each $v \in V_S$ is visited by some robot (i.e. there is a robot i with $p_t(i) = v$ for $0 \leq t \leq t'$), the vertex induced subgraph G_C^t is connected for $0 \leq t \leq t'$, and (iii) $r < n$.

In this definition 1^T is a string of 1s of length T . The reason for this definition is that we can provide a polynomial transformation from 3SAT¹ to d-CMPS to show that the problem is NP-hard (actually it is NP-complete). If the definition $(G_M, G_C, b, V_S, p_0, T)$ is used, a transformation has to check a sequence of at most T positions to derive a solution for the 3SAT instance. Such a check results in an exponential time transformation. Additionally, to verify whether a solution is a valid solution for a d-CMPS instance, it is necessary to iterate over exponential many positions, and therefore, the problem is not in NP then.

A. NP-HARDNESS RESULTS

To show the NP-hardness of d-CMPS, we provide a transformation from a 3SAT instance to a d-CMPS instance. The transformation for the 3SAT instance $\{c_1 = \{x_1, x_2, x_3\}, c_2 = \{\bar{x}_1, \bar{x}_2, x_4\}, c_3 = \{x_2, \bar{x}_3, \bar{x}_4\}\}$ is shown in Figure 2. The general transformation is described in the proof of

Proposition 3. *d-CMPS is NP-hard.*

Proof. The transformation from an instance of 3SAT with variables $W = \{x_1, \dots, x_\alpha\}$, and clauses $C = \{c_1, \dots, c_\beta\}$ to an instance of d-CMPS with graphs G_M and G_C with $n = 4\alpha + \beta + 2$ vertices is defined as:

- $r = \alpha + 1 < n$

¹A 3SAT instance consists of a set $W = \{x_1, \dots, x_\alpha\}$ of Boolean variables, and a set $C = \{c_1, \dots, c_\beta\}$ of clauses where each clause contains exactly three literals. The literals are of the form x_i or $\bar{x}_i$ where $x_i \in W$. The question is, whether there is an assignment of values from $\{True, False\}$ to the variables such that in every clause at least one literal evaluates to *True*.

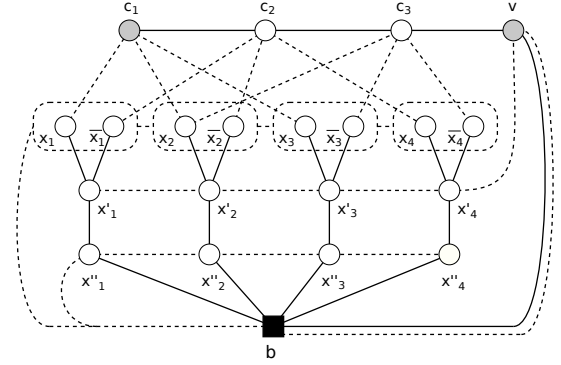


FIGURE 2. Example of a transformation from the 3SAT instance $\{c_1 = \{x_1, x_2, x_3\}, c_2 = \{\bar{x}_1, \bar{x}_2, x_4\}, c_3 = \{x_2, \bar{x}_3, \bar{x}_4\}\}$ to a d-CMPS instance. The filled circles depict sensing locations, the solid lines depict edges from E_M and the dashed lines depict edges from E_C . A dashed edge between two dashed boxes represents edges from E_C between all pairs of vertices from the two boxes. However, there is no edge from E_C between vertices within a dashed box.

- $V = \{x_1, \dots, x_\alpha, \bar{x}_1, \dots, \bar{x}_\alpha, x'_1, \dots, x'_\alpha, x''_1, \dots, x''_\alpha, c_1, \dots, c_\beta, b, v\}$
- $V_S = \{v, c_1\}$
- $[b, x''_i], [x''_i, x'_i], [x'_i, x_i], [x'_i, \bar{x}_i] \in E_M, 1 \leq i \leq \alpha$
- $[b, v], [v, c_\beta], [c_{i-1}, c_i] \in E_M, 2 \leq i \leq \beta$
- $[b, x''_i], [x''_{i-1}, x''_i] \in E_C, 2 \leq i \leq \alpha$
- $[v, x'_\alpha], [x'_{i-1}, x'_i] \in E_C, 2 \leq i \leq \alpha$
- $[b, z_1], [z_{i-1}, z_i] \in E_C, 2 \leq i \leq \alpha, z_i \in \{x_i, \bar{x}_i\}$
- $[c_j, z_i] \in E_C$ if z_i appears in $c_j, z_i \in \{x_i, \bar{x}_i\}$
- $p_0 = (v, x'_1, \dots, x'_\alpha)$
- $T = 2\beta$

On one hand, a solution with $WI = 2\beta - 1$ can be constructed from a solution of the 3SAT instance: A robot at vertex x'_i moves to $\bar{x}_i$ or x_i depending on the assignment of variable x_i . Since all clauses c_j are satisfied, robot 1 in v can move to c_1 and back to v . When the robot is in v , the other robots can move back to the x'_i vertices.

On the other hand, every solution that results in a $WI = 2\beta - 1$ must result in an assignment for the variables x_i that satisfies all clauses c_j . A robot can reach z_i only from its start position x'_i . If robot 1 moves to c_β , a robot in x'_i has to move to a z_i to keep robot 1 connected to the base station, and if robot 1 reaches c_1 , the positions of the other robots constitutes a satisfying assignment for the 3SAT instance. $\square$

The α vertices x''_i above the base station are not necessary to show the NP-hardness of d-CMPS. Nevertheless, the use of these vertices can show that the problem cannot be approximated with any constant factor unless $P = NP$. The structure of the vertices x'_i, x''_i and v prevents that multiple robots gather at some x'_i and move to some x_i and $\bar{x}_i$ simultaneously or let the robots change the assignment while robot 1 is commuting between c_1 and v , which would solve the problem without solving the 3SAT instance. No matter how large the time bound T is, commuting between c_1 and v can only be done by solving the 3SAT instance.

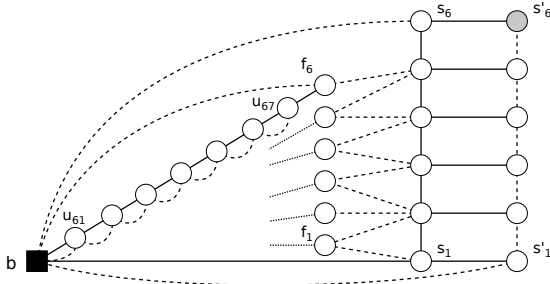


FIGURE 3. Example of a transformation from the SC instance $S = \{1, \dots, 5\}$, $F = \{f_1 = \{1, 2\}, f_2 = \{2\}, f_3 = \{2, 3\}, f_4 = \{3, 4\}, f_5 = \{4, 5\}, f_6 = \{5\}\}$ to a d-CMR instance. The filled circles depict sensing locations, the solid lines depict edges from E_M , the dashed lines depict edges from E_C . Only the path from b to f_6 is shown, the paths between b and f_1 through f_5 are indicated with a dotted line for better readability.

In practical scenarios a predefined number of robots starts at a base station, which raises two related questions: How many robots are at least necessary to reach a certain vertex, and can each sensing location be reached from the base station? We define the decision problem d-CMR to show that the problem of determining the minimum number of robots necessary to reach a single vertex (as well as determining the minimum number of time steps to do so), when all robots start at the base station, is NP-hard (NP-complete). The second question is treated in the following Subsection III-B.

Definition 4 (d-CMR). *Problem d-CMR is described by a set of tuples of the form $(G_M, G_C, b, g, r, 1^T)$ where g is the goal vertex and T is a time bound. The elements of the set have the following properties: (i) There is a sequence $(p_0, \dots, p_{t'})$, $t' \leq T$ such that $p_0 = (b, \dots, b)$, $p_{t'}(i) = g$ for some robot i , the vertex induced subgraph $G_C^{t'}$ is connected for $0 \leq t \leq t'$, and (ii) $r < n$.*

To show that d-CMR is NP-hard, we provide a transformation from set cover² (SC) to d-CMR. The transformation from an SC instance with elements $S = \{1, \dots, 5\}$ and subset family $F = \{f_1 = \{1, 2\}, f_2 = \{2\}, f_3 = \{2, 3\}, f_4 = \{3, 4\}, f_5 = \{4, 5\}, f_6 = \{5\}\}$ is shown in Figure 3. The general transformation is described in the proof of

Proposition 5. *d-CMR is NP-hard.*

Proof. The transformation from an instance of SC with elements $S = \{s_1, \dots, s_\alpha\}$, a subset family $F = \{f_1, \dots, f_\beta\}$, and a number k (the trivial case $k \geq \beta$ can be ignored) to an instance of d-CMR with $n = 1 + (M + 2)\beta + 2M$ is defined as (with $M := \max\{\alpha, \beta\}$):

- $r = M + k < M + \beta < n$
- $V = \{b, u_{11}, \dots, u_{1(M+1)}, \dots, u_{\beta 1}, \dots, u_{\beta(M+1)}, f_1, \dots, f_\beta, s_1, \dots, s_M, s'_1, \dots, s'_M\}$
- $[b, u_{i1}], [u_{i(M+1)}, f_i] \in E_M, 1 \leq i \leq \beta$
- $[u_{ij}, u_{i(j+1)}] \in E_M, 1 \leq i \leq \beta, 1 \leq j \leq M$

²An instance of SC consists of a set $S = \{s_1, \dots, s_\alpha\}$, a subset family $F = \{f_1, \dots, f_\beta\}$, $f_i \subseteq S$, and a number k . The question is, whether there are at most k subsets from F such that their union is equal to S .

- $[b, s_1], [s_j, s_{j+1}] \in E_M, 1 \leq j \leq M - 1$
- $[s_j, s'_j] \in E_M, 1 \leq j \leq M$
- $[b, u_{i1}], [b, f_i] \in E_C, 1 \leq i \leq \beta$
- $[u_{ij}, u_{i(j+1)}] \in E_C, 1 \leq i \leq \beta, 1 \leq j \leq M$
- $[b, s'_1], [s'_j, s'_{j+1}] \in E_C, 1 \leq j \leq M - 1$
- $[b, s_M] \in E_C$
- $[f_j, s_i] \in E_C$ if $s_i \in f_j$
- $g = s'_M$
- $T = 2\beta(M + 1) + M + 1$

A solution of the SC instance with $S = f_{j_1} \cup \dots \cup f_{j_l}$ and $l \leq k$ results in the following solution for the instance of d-CMR: l robots are placed at vertices $f_{j_1}, \dots, f_{j_l}$ subsequently with help of M robots that build a chain along the vertices u_{j_1} to u_{j_M} on a path to a vertex f_j ($j \in \{j_1, \dots, j_l\}$) and return to b . The sequence for placing one robot at f_6 in the example in Figure 3 is given as follows. Note that only the positions for the $M + 1 = 7$ robots, which are required to reach f_6 , are shown:

```
(..., b, b, b, b, b, b, b, ...)
(..., u61, u61, u61, u61, u61, u61, u61, ...)
(..., u61, u62, u62, u62, u62, u62, u62, ...)
(..., u61, u62, u63, u63, u63, u63, u63, ...)
...
(..., u61, u62, u63, u64, u65, u66, u67, ...)
(..., u61, u62, u63, u64, u65, u65, f6, ...)
(..., u61, u62, u63, u64, u64, u64, f6, ...)
...
(..., b, b, b, b, b, b, f6, ...)
```

After l robots have been placed at $f_{j_1}, \dots, f_{j_l}$, each vertex s_i is connected to the base station (since the set S is covered) and can be occupied by a robot. In the last step all these M robots move to the vertices s'_i at the same time along the edges $[s_i, s'_i] \in E_M, 1 \leq i \leq M$.

There are M robots necessary to place a robot at a vertex f_j or to change the position of a robot from a vertex f_{j_1} to another vertex f_{j_2} . Therefore, at most $k - 1$ robots can be placed at vertices s'_1 to s'_{k-1} , and since $k < M$, the vertex s'_M cannot be reached by changing the positions of robots at vertices f_j . This means that before the last robot can move to the goal $s'_M = g$, all vertices s_1 to s_α must be connected to the base station. This is only possible if at most k robots have been placed at vertices $f_{j_1}, \dots, f_{j_l}$, and a solution for the SC instance is determined by the occupied vertices $f_{j_1}, \dots, f_{j_l}$ at that moment when another robot reaches the goal vertex. $\square$

A natural strategy for the reachability problem when $E_M \subseteq E_C$ is that a group of robots start at the base station and move on a path from the base station to the goal in G_M until the connectivity to the base station would break when moving further. Every time when this happens, a relay robot stays at the current vertex and the remaining robots continue moving towards the goal [17]. Determining the optimal relay positions such that a goal can be reached with a predefined number of robots on a predefined path to the goal is also NP-hard, which we show with a transformation from 3SAT to d-CMRD (see Proposition 7 and Figure 4) defined in

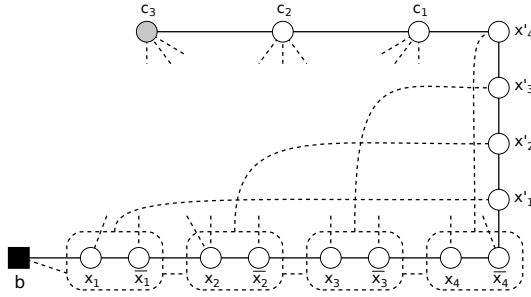


FIGURE 4. Example of a transformation from a 3SAT instance (same example as in Figure 2) to a d-CMRD instance. A dashed edge between two dashed boxes represents edges from E_C between all pairs of vertices from the two boxes. There is also an edge from E_C between the vertices within a dashed box (since $E_M \subseteq E_C$).

Definition 6 (d-CMRD). The problem d-CMRD is described by a set of tuples of the form (P, E_C, r) where $P := (b, v_1, \dots, v_n)$ is a sequence of vertices that describe a movement path, E_C denotes the connectivity edges between the vertices (at least $[b, v_1], [v_{i-1}, v_i] \in E_C, 2 \leq i \leq n$), and r is the number of robots. The elements of the set have the property that v_n can be reached with placing $r - 1$ robots at relay positions at some v_i .

Proposition 7. d-CMRD is NP-hard.

Proof. The transformation from an instance of 3SAT with variables $W = \{x_1, \dots, x_\alpha\}$, and clauses $C = \{c_1, \dots, c_\beta\}$ to an instance of d-CMPS with graphs G_M and G_C with $n = 3\alpha + \beta + 1$ vertices is defined as:

- $r = \alpha + 1$
- $P = (b, x_1, \bar{x}_1, \dots, x_\alpha, \bar{x}_\alpha, x'_1, \dots, x'_\alpha, c_1, \dots, c_\beta)$
- $[\bar{x}_\alpha, x'_1], [x'_{i-1}, x'_i] \in E_C, 2 \leq i \leq \alpha$
- $[x'_\alpha, c_1], [c_{i-1}, c_i] \in E_C, 2 \leq i \leq \beta$
- $[b, x_1], [\bar{x}_{i-1}, x_i] \in E_C, 2 \leq i \leq \alpha$
- $[x_i, \bar{x}_i] \in E_C, 1 \leq i \leq \alpha$
- $[z_{i-1}, z_i] \in E_C, 2 \leq i \leq \alpha, z_i \in \{x_i, \bar{x}_i\}$
- $[z_i, x'_i] \in E_C, 1 \leq i \leq \alpha, z_i \in \{x_i, \bar{x}_i\}$
- $[c_j, z_i] \in E_C$ if z_i appears in $c_j, z_i \in \{x_i, \bar{x}_i\}$

A solution of the 3SAT instance defines the positions of the α relays such that the robot $\alpha + 1$ can reach the goal vertex c_β .

If a robot can reach c_β , then the vertices $x'_1, \dots, x'_\alpha, c_1, \dots, c_\beta$ are connected to the base station, which is only possible if a relay is placed at each z_i such that the 3SAT instance is satisfied. $\square$

B. NOTE ON GRAPH TRAVERSAL

A precondition for the existence of a solution for a persistent surveillance problem (where the robots start at the base station) is that all sensing locations can be reached from the base station with the available number of robots. In the previous subsection we have shown that determining the minimum number of time steps to reach a goal vertex from the base station is NP-hard. In this subsection we argue that determining the minimum number of robots necessary to

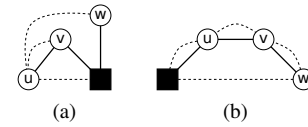


FIGURE 5. Simple examples for graph traversals. The graph in (a) cannot be traversed although G_M and G_C are connected. The graph in (b) can be traversed with two but not with one robot.

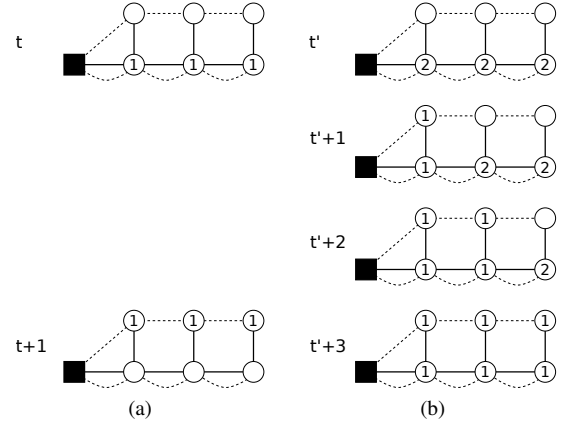


FIGURE 6. Illustration for the proof of Proposition 8. The graph in (a) is traversed by three robots in two time steps (from top to bottom), which move from the bottom to the top vertices simultaneously. This sequence corresponds to solution P in the text. In (b) the order in which the vertices are marked by Algorithm 1 are shown in four steps from top to bottom, which can be interpreted as traversal with 6 robots. This sequence corresponds to solution P' in the text. The numbers in the vertices indicate the number of robots that are at a vertex at a particular time step.

reach a goal vertex is also NP-hard. We do not attempt to determine a solution with the minimum number of robots but provide a necessary and sufficient condition for the existence of a solution with $n - 1$ robots, since there are graphs for which no solution exists even if G_M and G_C are connected.

We say that a graph can be traversed with r robots if there is a solution such that every sensing location can be visited with r robots starting at the base station. Although a graph cannot be traversed if sensing locations and the base station are at different connected components of G_M or G_C , the fact that G_M and G_C are connected, is not a sufficient condition for a graph to be traversable. Such an example is shown in Figure 5a where both G_M and G_C are connected but the graph cannot be traversed. If a robot moves from the base station to vertex v or w (because $[b, v] \in E_M$ and $[b, w] \in E_M$), it will be disconnected from the base station. The graph in Figure 5b can be traversed with two robots but not with one (although the distance in G_C is $\text{dist}_{G_C}(b, w) = 1$). An example sequence for traversal is $p_0 = (b, b), p_1 = (u, u), p_2 = (u, v), p_3 = (u, w)$.

Algorithm 1 determines whether a graph can be traversed with $n - 1$ robots. We show that it terminates with $\text{traverse} = \text{true}$ if and only if the graph can be traversed with $n - 1$ robots. Although using $n - 1$ robots is a trivial solution, determining a solution with the minimum number of robots such that a graph can be traversed also solves the

Algorithm 1 Traverse**Input:** V_S, G_M, G_C, b **Output:** $parent(\cdot), traverse$

```

1:  $parent(v) \leftarrow null, c \leftarrow false, \forall v \in V$ 
2:  $marked(b) \leftarrow true$ 
3:  $added \leftarrow true$ 
4: while  $added$  do
5:    $added \leftarrow false$ 
6:   for  $v \in V$  do
7:     if  $\neg marked(v)$  then
8:        $p \leftarrow null, c \leftarrow false$ 
9:       if  $\exists [v, w] \in E_M$  s.t.  $marked(w)$  then
10:         $p \leftarrow w$ 
11:       if  $\exists [v, u] \in E_C$  s.t.  $marked(u)$  then
12:         $c \leftarrow true$ 
13:       if  $p \neq null$  and  $c$  then
14:         $marked(v) \leftarrow true$ 
15:         $parent(v) \leftarrow p$ 
16:         $added \leftarrow true$ 
17:  $traverse \leftarrow |\{v \in V_S : marked(v)\}| = |V_S|$ 

```

SC instance of d-CMR. This holds because a solution with a minimum number of r^* robots corresponds to a set cover of cardinality $r^* - M$: If the goal s'_M can be reached with r^* robots, then M robots are necessary for the chain $s'_1, \dots, s'_M$, and $r^* - M$ robots are at some vertices f_i such that all $s_1, \dots, s_M$ are connected to b .

Proposition 8. *Algorithm 1 terminates with $traverse = true$ if and only if the graph can be traversed with $n - 1$ robots.*

Proof. First we show that if v is marked, then v can be visited. This can be shown by induction with the assumption that every vertex v that is added is occupied by a robot i that started at the base station (i.e. $p_0(i) = b$) and has connectivity to the base station (there is a path in G_C^t from b to v). For the first vertex v that is added there must be edges $[b, v] \in E_M$ and $[b, v] \in E_C$, i.e. robots can move from b to v . For subsequently marked vertices v there is an edge $[v, w] \in E_M$ such that a subset of the robots that are at w can move to v . The connectivity is maintained by a robot at vertex u and the edge $[v, u] \in E_C$. The traversal can be interpreted as a solution with $n - 1$ robots: If a vertex v is marked, a number of robots (sufficient to visit all vertices that are marked in subsequent steps) move from w to v such that there is exactly one robot at each v that has been marked when the algorithm terminates. The number of robots that should move to a marked vertex can be determined by traversing the tree described by the $parent$ function, which is not shown here.

To show that if a solution P with r robots traverses a graph, the algorithm terminates with $traverse = true$, a solution P' with $n - 1$ robots can be constructed with help of the following observation. From two subsequent positions p_t and p_{t+1} from P , where k robots change their position, a sequence of at most $k + 1$ positions $p'_{t'}, \dots, p'_{t'+l}$ with $l \leq k$ and $\{p_t(1) \cup \dots \cup p_t(r)\} \subseteq \{p'_{t'}(1) \cup \dots \cup p'_{t'+l}(n)\}$ such

that $\{p_{t+1}(1) \cup \dots \cup p_{t+1}(r)\} \subseteq \{p'_{t'+l}(1) \cup \dots \cup p'_{t'+l}(n)\}$ can be constructed. Figure 6a shows two steps of a solution P with $r = 3$ where the robots move at the same time from the lower to the upper vertices. In Figure 6b four steps of a solution P' with $r = n - 1 = 6$ are shown, which visits the same set of vertices as P . First, a robot j_1 in P' can move from $p'_{t'}(j_1) = p_t(i_1)$ to $p'_{t'+1}(j_1) = p_{t+1}(i_1)$ for a robot i_1 in P for which $[p_{t+1}(i_1), b] \in E_C$. In particular, $p_{t+1}(i_1)$ is marked, and these are the robots that move to the upper left vertex in Figures 6a and 6b. Then, for a robot i_2 with $[p_{t+1}(i_1), p_{t+1}(i_2)] \in E_C$ a robot j_2 can move in P' such that $p'_{t'+2}(j_2) = p_{t+1}(i_2)$. These are the robots that move to the upper middle vertex in Figure 6. This procedure can be continued for all robots resulting in $p'_{t'+l}(j_l) = p_{t+1}(i_l)$.

Furthermore, we have to show that Algorithm 1 marks all vertices that can be visited by solution P . Suppose that P traverses the graph but Algorithm 1 terminates with $traverse = false$. Then there must be two movement steps in P , p_t and p_{t+1} , such that $\{p_t(1) \cup \dots \cup p_t(r)\} \subseteq \{v \in V : marked(v)\}$ (this is certainly true for $t = 0$) and $\{p_{t+1}(1) \cup \dots \cup p_{t+1}(r)\} \not\subseteq \{v \in V : marked(v)\}$. However, this contradicts the observation above. $\square$

IV. PERSISTENT SURVEILLANCE ON CONVEX GRID AREAS

We consider a convex mission area without obstacles, which is divided into a two-dimensional grid of square cells with unit side length. A subset of these cells are sensing locations and the base station is located at a particular cell. A robot can move from a cell to one of the eight neighboring cells (except for cells at the boundary of the area) within one time step. The cells correspond to vertices of G_M and G_C , and E_M contains the edges between neighboring cells. The edges in E_C are defined by the communication range R^{com} , which is measured in cells. For example, $R^{com} = 3$ means that two cells are in communication range if the Euclidean distance between the center of the cells is smaller than or equal to 3. For the sake of completeness we recap the algorithms for short horizon (SH) and short horizon cooperative (SHC) movement planning [16].

A. SHORT HORIZON (SH) MOVEMENT PLANNING

The policy in [12] assigns a sensing location to each robot i at each time step t based on a weighted combination of the instantaneous idleness $I_t(v)$ of sensing location v , the distance between the robot and the sensing location $dist_{G_M}(p_t(i), v)$, and the minimum distance between v and any other robot $j \neq i$:

$$\begin{aligned}
 A(v, i) = & I_t(v) \\
 & + \omega_0 dist_{G_M}(p_t(i), v) \\
 & + \omega_1 \min_{j \neq i} \{dist_{G_M}(p_t(j), v)\}. \quad (2)
 \end{aligned}$$

Each robot gets assigned to the sensing location with the highest value $A(i, v)$ individually. The weighting parameters

Algorithm 2 Short horizon (SH)**Input:** $T, R, V_S, G_M, G_C, b, \omega_0, \omega_1$ **Output:** $P = (p_0, \dots, p_T)$

```

1:  $p_0 \leftarrow (b, \dots, b)$ 
2: for  $t \leftarrow 0, \dots, T - 1$  do
3:    $p' \leftarrow p_t$ 
4:    $A \leftarrow \text{CALCASSIGNMENTMATRIX}$ 
5:   for  $i \in R$  do
6:      $\text{goal}(i) \leftarrow \arg \max_{v \in V_S} \{A(v, i)\}$ 
7:      $p'(i) \leftarrow \arg \min_{v \in N(p_t(i))} \{ \text{dist}_{G_M}(v, \text{goal}(i)) : \text{ } \}$ 
        $G_C \langle \{p'(j) : j \in R \setminus \{i\}\} \cup \{v\} \cup \{b\} \rangle \text{ is connected} \}$ 
8:    $p_{t+1} \leftarrow p'$ 

9: procedure CALCASSIGNMENTMATRIX
10:  for  $v \in V_S, i \in R$  do
11:     $A(v, i) \leftarrow I_t(v) + \omega_0 \text{dist}_{G_M}(p_t(i), v) + \omega_1 \min_{j \neq i} \{ \text{dist}_{G_M}(p_t(j), v) \}$ 
12:  return  $A$ 

```

ω_0 and ω_1 are determined by an offline optimization algorithm where the parameter space is sampled and the mission is simulated to get the objective value for a particular set of parameters. We adopt this approach to enforce the connectivity constraints by disallowing moves that would result in a disconnected network and denote it as short horizon (SH) movement planner (Algorithm 2). The algorithm calculates positions for the robots starting at the base station over a finite horizon of T time steps. After calculating the assignment matrix according to Equation 2 (line 4), the goal for each robot is determined (line 6). The position of a robot i at time $t + 1$ is the neighboring position of the position at time t that is closest to the goal and does not disconnect the network of the robots (line 7). $N(p_t(i))$ denotes the set of neighbor positions of the current position $p_t(i)$ of robot i including the current position. The goals and new positions for time t are calculated for each robot consecutively in arbitrary order.

B. SHORT HORIZON COOPERATIVE (SHC) MOVEMENT PLANNING

Because the goals are assigned independently to the robots by the SH algorithm, it can happen that robots block each other infinitely due to the connectivity constraint. To overcome this mutual blocking problem, we developed an extension to SH based on graph matching and formation reconfiguration. SH assigns a goal to every robot, which then approaches its goal individually. In contrast to that, an iteration of SHC (Algorithm 3) consists of three phases: (i) goal selection, (ii) goal assignment and (iii) reconfiguration.

1) Goal selection

The goal selection phase starts with calculating an assignment matrix according to Equation 2 (line 3). In the next step, the sensing locations with the highest values together with the base station are selected as terminal vertices for

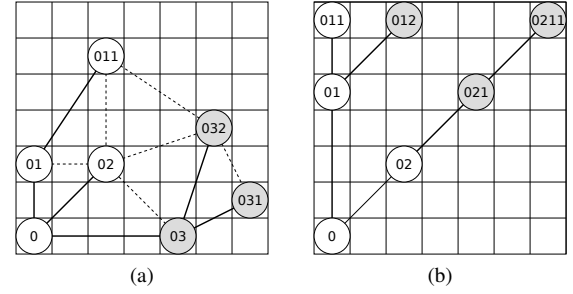


FIGURE 7. Outcome of the prefix labeling and graph matching algorithm for a grid area and 6 robots. The actual graph (a) models the state of the robots and the base station (node '0') at a particular time step with a line between two vertices if they are within communication range. A dashed line indicates the communication link and a solid line indicates the selected tree T_a in the actual graph G_a . The graph (b) models the final desired robot configuration T_f , where each robot is assigned to a relay position or a sensing location. Assume for example, that robots labeled with '011', '012', and '0211' are assigned to sensing locations (terminal vertices in the Steiner tree) but can reach them only with help of relays labeled with '01', '02', and '021' (nonterminal vertices). The shaded vertices are the extra vertices in (a) and the missing vertices in (b), respectively.

a Steiner tree in G_C . A Steiner tree is a vertex induced subgraph $G_C \langle \{b, v_1, \dots, v_i\} \cup N \rangle$ containing the terminal vertices $\{b, v_1, \dots, v_i\}$ and possibly a set of non-terminal vertices N such that the tree is connected. The function *calc_steiner_tree* calculates an approximate solution T_f for the NP-hard Steiner tree problem with minimum number of non-terminal vertices [53]. The algorithm tries to include as many sensing locations as possible into a Steiner tree that can be built with the available number of robots (lines 7-11).

2) Goal assignment

The goal assignment phase performs a matching between the desired final tree T_f and the actual vertex induced subgraph $G_a = G_C \langle \{p_t(i) : i \in R\} \cup \{b\} \rangle$ containing the positions of the robots at time t and the base station with the aim to reduce the time to reach the final configuration. A prefix labeling is applied to the vertices of the final tree T_f [36], which results in the labels L_f (line 12). Then a tree T_a is selected from G_a [54], and a matching between T_a and T_f is calculated [55] (line 14). The labels for the vertices that cannot be matched are determined (line 15). The extra and missing vertices [36] are computed in line 16. Extra vertices represent robots and are vertices with labels from L_a that are not in L_f ($\mathcal{V}_e$). Missing vertices represent vertices from V (final robot positions) with labels in L_f that are not in L_a ($\mathcal{V}_m$). Finally, a matching between extra and missing vertices is computed based on the distance in T_a (lines 17-19). Figure 7 depicts an example for the prefix labeling and graph matching algorithm.

3) Reconfiguration

In this phase every extra robot $i \in \mathcal{V}_e$ makes a move in G_M towards the position $M(i)$ according the matching M (line 25). Similar to line 7 in SH, the next position is chosen such that the network does not get disconnected. When a

Algorithm 3 Short horizon cooperative (SHC)**Input:** $T, R, V_S, G_M, G_C, b, \omega_0, \omega_1, \kappa$ **Output:** $P = (p_0, \dots, p_T)$

```

1:  $p_0 \leftarrow (b, \dots, b), t \leftarrow 0$ 
2: while  $t \leq T - 1$  do
3:    $A \leftarrow \text{CALCASSIGNMENTMATRIX}$ 
4:   for  $i \in R$  do
5:      $goal(i) \leftarrow \arg \max_{v \in V_S} \{A(v, i)\}$ 
6:      $value(i) \leftarrow \max_{v \in V_S} \{A(v, i)\}$ 
7:      $(v_1, \dots, v_r) \leftarrow \text{sort } goal(\cdot) \text{ according to } value(\cdot) \text{ (de-}$ 
       $\text{scending)}$ 
8:     for  $i \leftarrow 1, \dots, r$  do
9:        $T'_f \leftarrow \text{calc\_steiner\_tree}(G_C, \{b, v_1, \dots, v_i\})$  (see
[53])
10:      if  $|V(T'_f)| > (r + 1)$  then break
11:       $T_f \leftarrow T'_f$ 
12:       $L_f \leftarrow \text{label\_trie}(T_f)$  (see [36])
13:       $G_a \leftarrow G_C \setminus \{p_t(i) : i \in R\} \cup \{b\}$ 
14:       $(T_a, L_a) \leftarrow \text{calc\_graph\_matching}(G_a, T_f, L_f)$  (see
[54], [55])
15:       $L_a \leftarrow \text{complete\_labels\_trie}(T_a, L_a)$  (see [36])
16:       $(V_e, V_m) \leftarrow \text{calc\_extra\_missing}(L_a, L_f)$  (see [36])
17:      for  $i \in V_e, j \in V_m$  do
18:         $D(i, j) \leftarrow \text{dist}_{T_a}(i, \text{closest parent of } j \text{ also in } T_a)$ 
19:       $M \leftarrow \text{calc\_matching}(D)$ 
20:      for  $i \in V_e$  do  $sp(i) \leftarrow \text{shortest\_path}_{T_a}(i, M(i))$ 
21:      while true do
22:         $p' \leftarrow p_t$ 
23:        for  $i \in R$  do
24:          if  $i \in V_e$  then
25:             $p'(i) \leftarrow \text{first step in } G_M \text{ along } sp(i)$ 
26:            if  $p'(i) = M(i)$  then
27:               $L_a \leftarrow \text{update\_labels}(L_a)$ 
28:               $V_e \leftarrow V_e \setminus \{i\}$ 
29:             $G_a \leftarrow G_C \setminus \{p'(i) : i \in R\} \cup \{b\}$ 
30:            if compare\_graphs}(G_a, L_a, T_f, L_f) then
31:               $w \leftarrow v \in V_S \text{ with } L_f(v) = L_a(i)$ 
32:               $p'(i) \leftarrow \text{first step on}$ 
                 $\text{shortest\_path}_{G_M}(p'(i), w)$ 
33:           $p_{t+1} \leftarrow p', t \leftarrow t + 1$ 
34:          if all }  $v_1$  to  $v_\kappa$  have been reached then break

```

robot reaches the goal position, the corresponding label in L_a is updated to meet the label in L_f , and the robot is removed from the set of extra vertices (lines 26-28). If the tree T_f with the labels L_f is a subgraph of the actual connectivity graph G_a with the labels L_a , robot i with label $L_a(i)$ gets assigned to the sensing location v with the same label $L_f(v)$. Then the robot approaches its goal similar as in line 7 of SH, (lines 29-32). Finally, if all sensing locations $v_1, \dots, v_\kappa$ have been reached, the algorithm continues with the next goal selection phase (line 34).

C. FULL HORIZON (FH) MOVEMENT PLANNING

The full horizon approach requires a tour through all sensing locations. A leader robot traverses this path while the other robots maintain a chain to the base station in G_C . When a relay robot is at a sensing location, it is considered visited

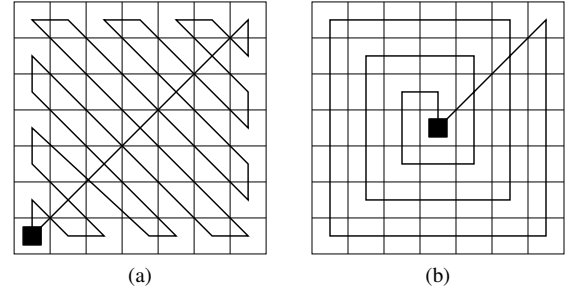


FIGURE 8. Two possible coverage tours for a convex area when the base station is at a corner (a) or near the center of the area (b). Both tours start at the base station, move first to the upper right sensing location but differ in how they cover the area towards the base station.

and can be skipped from the leading robot's tour by taking a shortcut to the next unvisited sensing location on the tour. If a considerable proportion of the area consists of sensing locations, it is beneficial that the tour visits sensing location first that are farther away from the base station and that the relaying robots maintain an equal distance between each other on the chain. Two possible tours are depicted in Figure 8. For persistent surveillance the tour traversal is repeated.

V. EXTENSION FOR PARTITIONED AREAS

The strategies presented in Section IV work best when the area is convex and free of obstacles. In arbitrary shaped environments the concurrent tree traversal (TT) approach from [17] can be used. In this approach a Steiner tree is generated that contains all sensing locations. The robots start at the root (base station) and traverse the edges placing relays when necessary. Branches are traversed concurrently if possible, i.e. robots split into groups at branching points. After the leaves of a branch have been visited, the robots retreat to the branching point and wait for other groups traversing other branches to recombine the groups if a split happened. As shown by Mosteo and Montano [17], the problem of determining how to split the group of robots to explore the tree as fast as possible is NP-hard, and different heuristic strategies for determining when (*splitting strategy*) and how (*selection strategy*) to split are discussed. The *early split* strategy splits the group at a branching point as soon as all leaves in subsequent branches, that have not been traversed yet, can be reached with the available number of robots. The *late split* strategy splits the robots when all leaves in the subsequent branches, that have not been traversed yet, can be visited concurrently. When no split happens, the next branch has to be selected at a splitting point. The *far selection* strategy selects the branch with the farthest leaf, and the *near selection* strategy selects the branch with the nearest leaf. In total there are four different combinations of strategies to traverse a tree.

The direct application of the tree traversal approach requires a tree that contains all sensing locations. To combine the advantages of the tree traversal with convex area coverage, we consider a partitioning of the area into convex

partitions where the areas are the vertices of the tree, and we assume that a partitioning of the area is given (e.g. determined by some algorithm described in [56], [57]). For each partition a cell is defined which is the release point for the robots covering that partition. The release points are connected with edges that form a tree (to be determined after partitioning), and two numbers are associated with each edge: how many relays are necessary to build a chain between the release points (which is the number of cells on the shortest path in G_C and an optimal solution of CMRD on a grid), and how long it takes to travel between the release points (which is the number of cells on the shortest path in G_M).

The problem in [17] is different from the problem discussed here in two aspects. First, we consider partitions and not only single vertices. Each release point is considered as own branch to which a number of robots has to be assigned for covering the partition. To cover a partition, a certain minimum number of robots is necessary due to the limited communication range. Second, since we consider persistent surveillance, the groups do not have to retreat to the base station and recombine there if all partitions up to a certain depth in the tree are already covered concurrently.

Our strategy works as follows. The robots start at the base station and move along an edge to a release point, dropping relays when necessary. When the robots reach a release point, one relay stays at the release point and the remaining robots are split into subgroups according to the same strategies as described above. The robots assigned to a partition start to cover the area according to one of the algorithms described in Section IV and return to the release point after all sensing locations have been visited. Here, they cover the area again, wait and recombine with other groups to move to the next release point, or retreat to a release point at a lower depth in the tree. We formally define the problem as:

Definition 9 (d-CMPSTT). *The problem d-CMPSTT is defined by a set of tuples of the form $(A, B, \Gamma, \Delta, b, r, T)$. We denote V_P the set of partitions, $A : V_P \rightarrow \mathbb{N}_{\geq 0}$ is the minimum number of robots necessary to cover a partition, $B : (V_P \times V_P) \rightarrow \mathbb{N}_{\geq 0}$ is the number of relays necessary between two release points, $\Gamma : (V_P \times \mathbb{N}_{>0}) \rightarrow \mathbb{N}_{\geq 0}$ is the time it takes to cover a partition with a certain number of robots, $\Delta : (V_P \times V_P) \rightarrow \mathbb{N}_{\geq 0}$ is the travel time between two release points. The vertex $b \in V_P$ is the partition containing the base station. The tree is given by the functions B and Δ . The number of robots is r , and T is a time bound for WI .*

A solution of a CMPSTT instance is the information about how the robots should split at each release point, which includes the branches visited concurrently, the order of the branches in the case not all can be visited concurrently, and how many robots are assigned to each branch. An example is shown in Figure 9. Assume $A(v_i) = 2, A(b) = 0, B \equiv 0, \Gamma(v_i) = 0, \Gamma(b) = 0, \Delta \equiv 1$. With $r = 6$ a possible solution would be a splitting $((v_1, v_4))$ at the base station with the assigned number of robots $((4, 2))$. This means, that branch v_1 and v_4 get visited concurrently and that 4

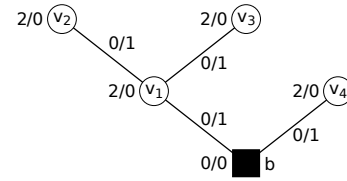


FIGURE 9. Example of a tree for CMPSTT. The numbers next to the vertices show A/T , and the numbers next to the edges represent B/Δ .

robots are assigned to branch v_1 , and 2 robots are assigned to branch v_4 . Since there are no more branches at the base station, (v_1, v_4) is the only tuple in this splitting. A splitting in v_1 could be $((v_1, v_2), (v_1, v_3))$, where branches v_1 and v_2 are visited concurrently before the robots recombine at v_1 and then visit v_1 and v_3 concurrently. After recombination at v_1 , this cycle starts again. The number of robots assigned to the branches are $((2, 2), (2, 2))$. In this solution the robots assigned to v_1 and v_4 at the base station do not have to retreat and combine at the base station, since the branches v_1 and v_4 are visited concurrently and there are no sensing locations in the base station partition, which results in $WI = 6$ (which is the idleness for v_2 and v_3). In this notation of a splitting it is allowed that a branch occurs multiple times in a splitting (e.g. at splitting point v_1). To be able to enumerate a splitting explicitly and prevent an infinite sequence of tuples in a splitting, we restrict the occurrences of branches in a splitting to the number of branches at the splitting point.

Mosteo and Montano [17] show by a reduction from multi-processor scheduling that the problem of determining the order in which the branches should be visited is NP-hard. Since this is a special case of CMPSTT (with $A(p) = 1, \forall p \in V_P$), CMPSTT it is NP-hard as well. Nevertheless, we can show that CMPSTT with $A(p) > 1$ for some p is also NP-hard. We show this with a reduction from the number partition problem³. The details of the reduction are shown in the proof of

Proposition 10. *d-CMPSTT is NP-hard.*

Proof. An instance of a number partition problem can be transformed into an instance of d-CMPSTT (the edges E_A of the tree are enumerated explicitly for convenience)

- $V_P = \{b, v_1, \dots, v_\alpha\}$,
- $E_A = \{[b, v_i] : i = 1, \dots, \alpha\}$
- $A(b) = 0, A(v_i) = s_i, i = 1, \dots, \alpha$
- $B(b, v_i) = 0, i = 1, \dots, \alpha$
- $\Gamma(b, \cdot) = 0, \Gamma(v_i, \cdot) = 1, i = 1, \dots, \alpha$
- $\Delta(b, v_i) = 1, i = 1, \dots, \alpha$
- $r = 1/2 \sum_{i=1, \dots, \alpha} s_i$
- $T = 6$

A solution to the number partition problem gives a solution to d-CMPSTT with $WI = 6$: Let the subsets be ordered such

³The number partition problem is defined as the problem of finding a partition of a set $S = \{s_1, \dots, s_\alpha\}, s_i \in \mathbb{N}_{>0}$, into two subsets S_1 and S_2 such that the sum of the elements in S_1 is equal to the sum of the elements in S_2 .

that $\sum_{i=1,\dots,k} s_i = r = 1/2 \sum_{i=1,\dots,\alpha} s_i$ with $k < \alpha$. Then the robots visit the partitions $v_1, \dots, v_k$ concurrently starting at the base station, retreat to the base station and visit the partitions $v_{k+1}, \dots, v_\alpha$. After the robots retreat again, this cycle starts from the beginning. The number of robots assigned to each partition is determined by $A(v_i) = s_i, i = 1, \dots, \alpha$.

Assume that there is a solution to d-CMPSTT that results in $WI = 6$, i.e. each partition gets visited once or twice in each cycle since $r < \sum_{i=1,\dots,\alpha} A(v_i)$, and the idleness of each partition is smaller or equal to 6. Assume also that this solution cannot be converted to a solution to the number partition problem. Then the following two situations can occur: (i) $WI = 3$ for some partitions, i.e. it gets visited twice in each cycle, or (ii) the number of robots assigned to a branch v_i is larger than $A(v_i) = s_i$.

In the first case $v_1, \dots, v_k$ are the partitions that get visited only in the first half of the cycle, $v_{k+1}, \dots, v_l$ are the partitions that get visited only in the second half of the cycle, and $v_{l+1}, \dots, v_\alpha$ are the partitions that get visited twice in each cycle. It follows that

$$\begin{aligned} \sum_{i=1}^k A(v_i) + \sum_{i=l+1}^{\alpha} A(v_i) &\leq r, \text{ and} \\ \sum_{i=k+1}^l A(v_i) + \sum_{i=l+1}^{\alpha} A(v_i) &\leq r, \text{ and therefore} \\ \frac{1}{2} \sum_{i=1}^l A(v_i) + \sum_{i=l+1}^{\alpha} A(v_i) &\leq r, \end{aligned}$$

which can only be the case if the second sum in the last equation is zero, which is a contradiction to the definition of the number partition problem, or there are no partitions which get visited twice, which is a contradiction to the assumption.

In the second case, a similar argument holds:

$$\begin{aligned} \sum_{i=1}^k A(v_i) + r_1 &\leq r, \text{ and} \\ \sum_{i=k+1}^l A(v_i) + r_2 &\leq r, \text{ and therefore} \\ \frac{1}{2} \sum_{i=1}^l A(v_i) + \frac{r_1 + r_2}{2} &\leq r, \end{aligned}$$

where r_1 and r_2 are the surplus robots assigned to the branches. According to the assumption, more robots are assigned to the branches than necessary, and these robots are the surplus robots. From the last equation follows that r_1 and r_2 must be zero, which is a contradiction to the assumption. Therefore, a splitting for the d-CMPSTT instance has the form $((v_1, \dots, v_k), (v_{k+1}, \dots, v_\alpha))$, which constitutes a solution for the number partition problem. $\square$

VI. SIMULATION RESULTS

A first simulation is conducted on a grid of 30×30 cells with the base station at the lower left corner and four sensing

locations: s_1 at cell coordinates $(30, 1)$, s_2 at $(30, 30)$, s_3 at $(1, 30)$ and s_4 at $(15, 15)$. The communication range R^{com} is half of the diagonal diameter of the grid, and the number of robots $r = 3$. The results are shown in Figure 10. In this simple scenario SHC shows the optimal behavior resulting in the best possible WI of 60 (twice the side length of the area): One robot acts as relay for the other two and visits s_4 , while another robot is commuting between s_1 and s_2 and the third robot between s_2 and s_3 . SH and FH visit one sensing location after the other. Because the robots start at the base station, the instantaneous worst idleness WI_t grows until all sensing locations have been visited for the first time before it drops to WI . The optimal behavior of SHC does not sustain on larger scenarios as can be seen in subsequent simulations.

We conduct the remaining simulation studies on a grid of 30×30 cells with 900 sensing locations and with the base station at the lower left corner. The parameters ω_0 and ω_1 are determined with the pattern search algorithm provided by the Global Optimization Toolbox from Matlab. This algorithm systematically samples the parameter space and converges to a local minimum [58]. For each pair (ω_0, ω_1) polled by the pattern search algorithm, SH (Algorithm 2) or SHC (Algorithm 3) is executed for a predefined number of time steps T_O (optimization horizon), and the objective for the resulting solution $\pi = (p_0, \dots, p_{T_O})$ is evaluated. The objective value of a solution π is the time duration until all sensing locations have been visited at least once or T_O if not all sensing locations have been visited within the optimization horizon. This objective value corresponds to the coverage time CT and can be formally defined as $CT := \min\{\arg \min_{0 \leq \tau \leq T_O} \{S_\tau : V_S \subseteq S_\tau\} \cup \{T_O\}\}$, with $S_\tau := \bigcup_{0 \leq t \leq \tau} \{p_t(i) : i \in R\}$. To reduce the optimization time, SH and SHC are aborted in each run in the course of the optimization as soon as all sensing locations have been visited at least once. We set the optimization horizon T_O to 1800 time steps.

We set the value $\kappa \in \{1, \dots, r\}$ of SHC to $\lfloor r/2 \rfloor$, since this produces results comparable to the results when κ is included in the optimization with pattern search as well as to the results from a two-step approach. The optimization of $(\omega_0, \omega_1, \kappa)$ with pattern search results in an increased optimization time since the parameter space has an additional dimension, which leads to an increased number of polls (approximately by 50 to 100 percent). The two-step approach uses pattern search to determine the parameters (ω_0, ω_1) with $\kappa = \lfloor r/2 \rfloor$ in the first step. In the second step, the set $\{1, \dots, r\}$ is exhaustively searched for the best objective value with the parameters (ω_0, ω_1) from the first step.

Pattern search uses a mesh grid to poll points around a current center in each iteration [59]. We set the start point of the search to $x_0 = (0, 0)$, and pattern search polls the points $x_0, x_0 + (1, 0), x_0 + (0, 1), x_0 + (-1, 0), x_0 + (0, -1)$ in the first iteration with an initial mesh of size 1. The point x_1 with the lowest objective value is the center of the new mesh in the second iteration. If x_i is different from x_{i-1} , the mesh size is multiplied by 2, and the points

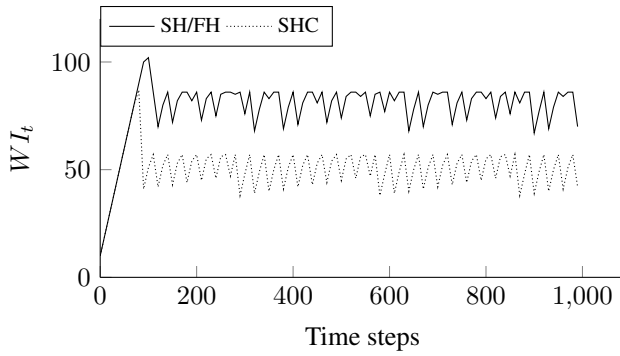


FIGURE 10. Comparison of the instantaneous idleness WI_t over time for a simple example with 4 sensing locations and 3 robots.

$x_1 + 2(1, 0), x_1 + 2(0, 1), x_1 + 2(-1, 0), x_1 + 2(0, -1)$ are polled in the second iteration. If none of the polled points has a better objective value than x_i , x_i becomes the center of a new mesh, which has the size of the old mesh multiplied by 0.5. This procedure continues until a user defined stopping criterion is met. We set the stopping criterion to a mesh tolerance of 0.01. This procedure can be parallelized, and we use 4 cores to evaluate polls around a mesh center in parallel.

In the following experiment we evaluate the performance of SH, SHC, FH and TT with an increasing number of robots r and a decreasing communication range R^{com} . The communication range has been chosen such that all r robots are necessary to reach the upper right sensing location in the area. We assume that r and R^{com} are determined by the available number of robots and by the environment, respectively (i.e. both parameters serve as input to the path planning problem). The purpose of this experiment is to assess the different algorithms on different scenarios. Figure 11 shows the coverage time CT , which is the objective value of the parameter optimization for SH and SHC as described above. CT can serve as an estimate of WI for comparing the different algorithms. Figure 11 shows that SH benefits from an increasing number of robots up to $r = 10$ but suffers from the low communication range with $r = 15$ and is not able to visit all sensing locations within the time horizon. For the TT algorithm a tree has been chosen that is the union of all the shortest paths from each sensing location to the base station. All four combinations of strategies described in Section V are tried and the best value is recorded. This tree is traversed basically in a depth-first order without any concurrency, resulting in a CT that is twice the number of sensing locations because in a tree traversal each cell in the tree is visited at least twice.

In the next experiment we set the communication range to a constant value of a quarter of the diameter of the area, such that 4 robots are necessary to reach all sensing locations. The results are shown in Figure 12. It shows that SH benefits most from an increasing number of robots and CT approaches the lower bound $|V_S|/r$. The reason for the drop of TT at $r = 8$ is the splitting and selection strategy. There are three branches originating at the base station, one which covers

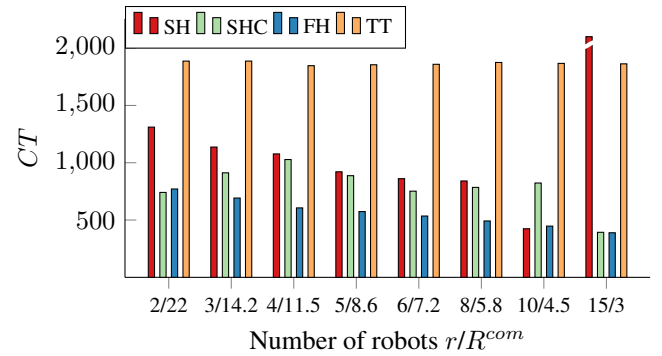


FIGURE 11. Comparison of CT with increasing number of robots r and decreasing communication range R^{com} . The broken bar for SH indicates that it was not able to visit all sensing locations within the provided time horizon for the optimization.

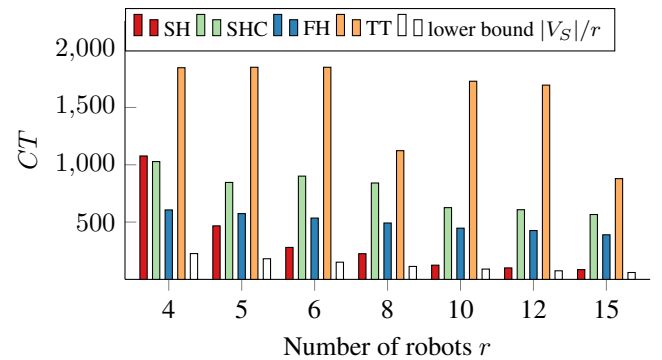


FIGURE 12. Comparison of CT with increasing number of robots r and fixed communication range R^{com} (a quarter of the diameter of the area).

mainly the lower right, one the diagonal, and one the upper right proportion of the area. With 6 and 8 robots all branches are visited subsequently, which results in a lower CT for 8 robots. With 10 robots the diagonal (which contains a small number of sensing locations) and lower right (which contains a larger number of sensing locations) branches are visited concurrently, which results in the situation that the robots visiting the diagonal proportion finish earlier and stay idle while waiting for recombination of the split group.

Figure 14 shows CT for an increasing number of robots r and a decreasing communication range R^{com} , where a number of robots are used as relays with fixed positions between the base station and a release point near the center of the area. The scenario for $r = 5$ (3 sensing robots) is shown in Figure 13a. The remaining robots start to cover the area from the release point. The purpose of this experimental setup is to assess whether the release point for a partitioning should be at the center or at the corner closest to the base station of a partition. The number of robots in total and the communication range is the same as in Figure 11. The numbers on the x-axis in Figure 14 are the numbers of robots that are available for coverage of the area. A longer chain of relays shifts the release point closer to the center and less robots are available for visiting sensing locations, which results in the expected degraded performance.

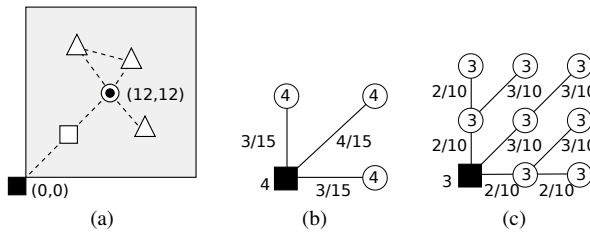


FIGURE 13. (a) Illustration of the scenario with 5 robots (of which are 3 sensing robots) for the results in Figure 14 (see Figure 1 for the meaning of the symbols). The numbers show the x- and y-coordinates of the cells of the base station and the release point. (b) Tree for CMPSTT for the results in Figure 16, and (c) Figure 17, respectively. The numbers in the vertices and next to the base station represent A , and the numbers next to the edges represent B/Δ of the CMPSTT instances. The base station is the release point of the lower left partition.

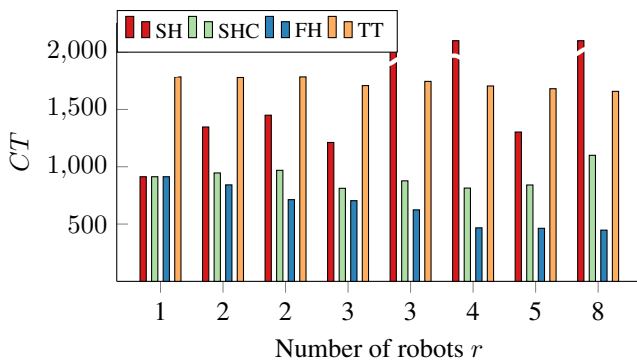


FIGURE 14. Comparison of CT with increasing number of robots r and decreasing communication range R^{com} with a certain number of relays between the base station and a release point near the center of the area. The number of robots in total and the communication range are the same as in Figure 11 for each scenario. The numbers on the x-axis are the numbers of robots that are available for coverage of the area.

Next, we investigate WI for area partitioning. We do not consider a non-convex area with obstacles explicitly but a regular partitioning of the convex area into convex subareas. This allows us to compare the performance of the unpartitioned case with the performance of the partitioned case, and we do not rely on an algorithm for dividing a non-convex polygon into a set of convex polygons. For non-cyclic algorithms WI can only be estimated over a limited time horizon. Figure 15 shows the results for the unpartitioned area for a time horizon of 3000 time steps. WI is calculated as the maximum time between two consecutive visits within the time period after the first and before the last visit of a sensing location in the horizon and over all sensing locations. For comparison with the tree traversal of partitions (CMPSTT), the area is partitioned into 2×2 (Figure 16) and 3×3 (Figure 17) rectangular partitions of equal size and traversed with different algorithms for convex partitions (SH, SHC, FH). The release point for each partition is the lower left corner of the partition (the base station is the release point of the lower left partition). Figure 13b and Figure 13c show the trees for CMPSTT. The parameters for SH and SHC have been determined for the whole area for the case of an

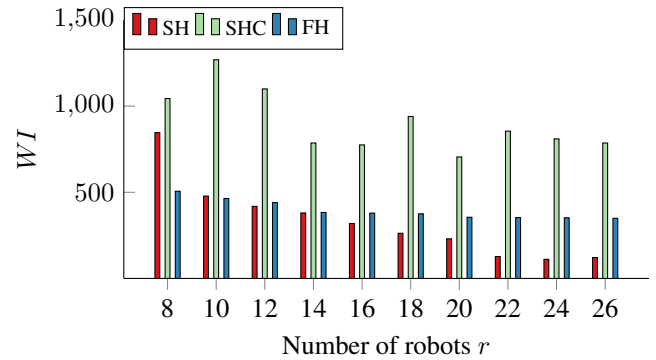


FIGURE 15. Comparison of WI with an increasing number of robots r and constant communication range $R^{com} = 5.8$ over a time horizon of 3000 time steps.

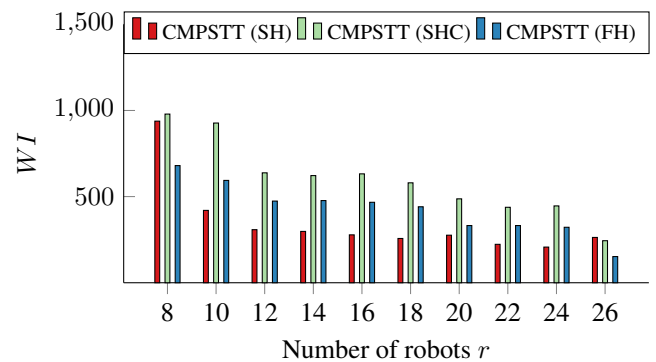


FIGURE 16. Comparison of WI with an increasing number of robots r and constant communication range $R^{com} = 5.8$ for the tree traversal of partitions (CMPSTT) of the area partitioned into 2×2 equally sized rectangular partitions and for different algorithms for the convex partitions (SH, SHC, FH).

unpartitioned area and for a single partition in the case of a partitioning. The optimization times for determining (ω_0, ω_1) have been recorded for each scenario and are shown in Figure 18 for SH. For the partitioning, the optimization times for the different number of robots assigned to a partition are summed up. For example, if $r_1, \dots, r_k$ represent the different number of robots assigned to any partition, then the optimization times for $r_1, \dots, r_k$ robots are summed up since all these different parameters (ω_0, ω_1) for different number of robots are required.

Although the performance is comparable between the unpartitioned and partitioned scenario up to 20 robots for SH, optimizing for smaller areas can greatly reduce the optimization time. As the number of robots gets larger, the number of robots assigned to partitions gets smaller because the partitions are covered concurrently, and therefore the optimization time decreases for an increasing total number of robots. The performance of SHC and FH can even be improved by partitioning the area. For these approaches using a smaller number of robots on a smaller area is more effective, because the performance improvement is smaller as compared to SH with an increasing number of robots (cf. Figure 12 and Figure 15).

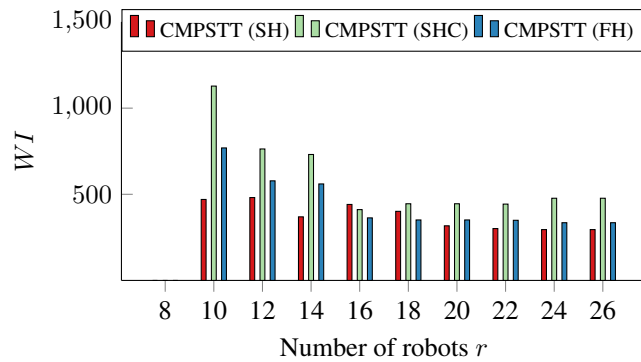


FIGURE 17. Comparison of WI with an increasing number of robots r and constant communication range $R^{com} = 5.8$ for the tree traversal of partitions (CMPSTT) of the area partitioned into 3×3 equally sized rectangular partitions and for different algorithms for the convex partitions (SH, SHC, FH). In this scenario it is not possible to reach all sensing locations with 8 robots.

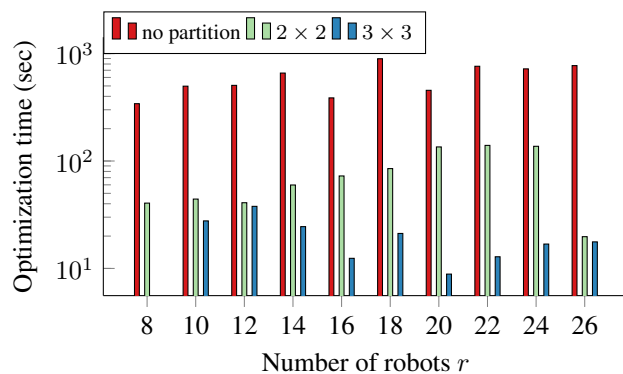


FIGURE 18. Optimization time for SH for an increasing number of robots r for the unpartitioned area and for partitions of different size.

VII. CONCLUSION

In this work we have investigated path planning for multiple robots for persistent surveillance with connectivity constraints. We introduce the problem and related problem instances on graphs, which have not been considered yet in literature, and show that they are NP-hard. We propose several strategies on grids that can be applied to convex areas. On one hand, the simple short horizon (SH) strategy, which selects the next goal of a robot greedily without considering the anticipated goals of other robots, achieves the best performance when the number of robots is larger than the minimum number of robots required to reach all sensing locations. In this case the additional robots can be used more effectively than in the short horizon cooperative (SHC) and full horizon (FH) approaches. On the other hand, if all robots are required to reach all sensing locations, SHC and FH perform better. Although FH performs better than SHC in most cases, SHC does not require a preplanned tour through all sensing locations.

The short horizon approaches SH and SHC rely on parameters controlling the goal selection that have to be optimized before the mission execution. To apply these strategies on more general environments, which arise from discretization

of real world scenarios, we propose a combination with a tree traversal approach. The simulation results on the considered scenarios indicate that the combination with tree traversal on a partitioned area does not impair the performance considerably up to a certain number of robots but reduces the optimization time substantially. With a larger number of robots the trade-off between performance and optimization time is revealed for SH. Partitioning can increase the performance for the other approaches, because using a smaller number of robots on smaller areas is more effective.

We use pattern search for finding locally optimal parameters that control the goal selection. The parameter that controls the start of a new iteration of goal selection, assignment and reconfiguration of SHC is fixed but can be part of the optimization with pattern search. The drawback is an increased optimization time. Except for SHC, using more robots results in an increased performance. Therefore, the number of robots can be considered as an input to the problem rather than an optimization parameter and depends on the available number of robots or the number of robots that can be handled practically. Depending on the algorithm, the shape and size of subareas of a partitioning of the area can be considered as optimization parameters, since for some algorithms a partitioning results in a performance that is better than the performance on the unpartitioned case. Nevertheless, determining optimal partitions is not in the scope of this work and a possible extension for future work.

The presented algorithms can be used to compute paths that have to be followed by the robots, which requires a central entity for computation and synchronization among all robots at each time step. This approach has several limitations, which can be subject of future extensions. If a robot fails, the network gets disconnected and the remaining robots can continue the mission only after failure recovery measures. Furthermore, synchronization among all robots (at least among the robots within a partition) requires a reliable network connection that ensures a timely delivery of the state of each robot through the network to the central entity. Other possible extensions are algorithms for the presented problems that can be applied to general or other special types of graphs (e.g. where $E_M \subseteq E_C$).

REFERENCES

- [1] M. Erdelj, E. Natalizio, K. R. Chowdhury, and I. F. Akyildiz, "Help from the sky: Leveraging UAVs for disaster management," *IEEE Pervasive Computing*, vol. 16, no. 1, pp. 24–32, Jan. 2017.
- [2] J. Scherer, B. Rinner, S. Yahyanejad, S. Hayat, E. Yanmaz, T. Andre, A. Khan, V. Vukadinovic, C. Bettstetter, and H. Hellwagner, "An autonomous multi-UAV system for search and rescue," in *Proceedings of the First Workshop on Micro Aerial Vehicle Networks, Systems, and Applications for Civilian Use - DroNet '15*, 2015, pp. 33–38.
- [3] A. Khan, B. Rinner, and A. Cavallaro, "Cooperative robots to observe moving targets: Review," *IEEE Transactions on Cybernetics*, vol. 48, no. 1, pp. 187–198, Jan. 2018.
- [4] K. A. Ghamry and Y. Zhang, "Cooperative control of multiple UAVs for forest fire monitoring and detection," in *Proceedings of the IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA)*. IEEE, Aug. 2016, pp. 1–6.
- [5] B. Liu, O. Dousse, P. Nain, and D. Towsley, "Dynamic coverage of mobile

- sensor networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 24, no. 2, pp. 301–311, Feb. 2013.
- [6] M. Rossi and D. Brunelli, "Autonomous gas detection and mapping with unmanned aerial vehicles," *IEEE Transactions on Instrumentation and Measurement*, vol. 65, no. 4, pp. 765–775, Apr. 2016.
 - [7] E. Masehian, M. Jannati, and T. Hekmatfar, "Cooperative mapping of unknown environments by multiple heterogeneous mobile robots with limited sensing," *Robotics and Autonomous Systems*, vol. 87, pp. 188–218, Jan. 2017.
 - [8] J. M. Santos, T. Krajnc, and T. Duckett, "Spatio-temporal exploration strategies for long-term autonomy of mobile robots," *Robotics and Autonomous Systems*, vol. 88, pp. 116–126, Feb. 2017.
 - [9] D. Portugal and R. P. Rocha, "Performance estimation and dimensioning of team size for multirobot patrol," *IEEE Intelligent Systems*, vol. 32, no. 6, pp. 30–38, Nov. 2017.
 - [10] F. Pasqualetti, A. Franchi, and F. Bullo, "On cooperative patrolling: Optimal trajectories, complexity analysis, and approximation algorithms," *IEEE Transactions on Robotics*, vol. 28, no. 3, pp. 592–606, Jun. 2012.
 - [11] F. Lauri, J.-C. Créput, and A. Koukam, "The multi-agent patrolling problem theoretical results about cyclic strategies," in *Lecture Notes in Computer Science*. Springer, 2014, vol. 8473, pp. 171–182.
 - [12] N. Nigam, S. Bieniański, I. Kroo, and J. Vian, "Control of multiple UAVs for persistent surveillance: Algorithm and flight test results," *IEEE Transactions on Control Systems Technology*, vol. 20, no. 5, pp. 1236–1251, Sep. 2012.
 - [13] C. Franco, G. Lopez-Nicolas, C. Sagues, and S. Llorente, "Persistent coverage control with variable coverage action in multi-robot environment," in *Proceedings of the 52nd IEEE Conference on Decision and Control*, Dec. 2013, pp. 6055–6060.
 - [14] J. Banfi, N. Basilico, and F. Amigoni, "Multirobot reconnection on graphs: Problem, complexity, and algorithms," *IEEE Transactions on Robotics*, vol. 34, no. 5, pp. 1299–1314, Oct. 2018.
 - [15] J. Scherer and B. Rinner, "Persistent multi-UAV surveillance with energy and communication constraints," in *Proceedings of the IEEE International Conference on Automation Science and Engineering (CASE)*, Aug. 2016, pp. 1225–1230.
 - [16] —, "Short and full horizon motion planning for persistent multi-UAV surveillance with energy and communication constraints," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Sep. 2017, pp. 230–235.
 - [17] A. R. Mosteo and L. Montano, "Concurrent tree traversals for improved mission performance under limited communication range," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Oct. 2009, pp. 2840–2845.
 - [18] J. Scherer and B. Rinner, "Multi-Robot Patrolling with Sensing Idleness and Data Delay Objectives," Jun. 2019. [Online]. Available: <http://arxiv.org/abs/1906.11539>
 - [19] —, "Persistent Multi-UAV Surveillance with Data Latency Constraints," Jul. 2019. [Online]. Available: <http://arxiv.org/abs/1907.01205>
 - [20] J. J. Acevedo, B. C. Arrue, J. M. Diaz-Banez, I. Ventura, I. Maza, and A. Ollero, "Decentralized strategy to ensure information propagation in area monitoring missions with a team of UAVs under limited communications," in *Proceedings of the International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, May 2013, pp. 565–574.
 - [21] S. L. Smith, M. Schwager, and D. Rus, "Persistent robotic tasks: Monitoring and sweeping in changing environments," *IEEE Transactions on Robotics*, vol. 28, no. 2, pp. 410–426, Apr. 2012.
 - [22] F. Pasqualetti, J. W. Durham, and F. Bullo, "Cooperative patrolling via weighted tours: Performance analysis and distributed algorithms," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1181–1188, Oct. 2012.
 - [23] D. Mitchell, M. Corah, N. Chakraborty, K. Sycara, and N. Michael, "Multi-robot long-term persistent coverage with fuel constrained robots," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, May 2015, pp. 1093–1099.
 - [24] V. Mersheeva and G. Friedrich, "Multi-UAV monitoring with priorities and limited energy resources," in *Proceedings of the International Conference on Automated Planning and Scheduling*, Apr. 2015.
 - [25] S. G. Manyam, S. Rasmussen, D. W. Casbeer, K. Kalyanam, and S. Manickam, "Multi-UAV routing for persistent intelligence surveillance & reconnaissance missions," in *Proceedings of the International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, Jun. 2017, pp. 573–580.
 - [26] J. Keller, D. Thakur, M. Likhachev, J. Gallier, and V. Kumar, "Coordinated path planning for fixed-wing uas conducting persistent surveillance missions," *IEEE Transactions on Automation Science and Engineering*, vol. 14, no. 1, pp. 17–24, Jan. 2017.
 - [27] X. Lan and M. Schwager, "Rapidly exploring random cycles: Persistent estimation of spatiotemporal fields with multiple sensing robots," *IEEE Transactions on Robotics*, vol. 32, no. 5, pp. 1230–1244, Oct. 2016.
 - [28] M. Ji and M. Egerstedt, "Distributed coordination control of multiagent systems while preserving connectedness," *IEEE Transactions on Robotics*, vol. 23, no. 4, pp. 693–703, Aug. 2007.
 - [29] M. Schuresko and J. Cortés, "Distributed tree rearrangements for reachability and robust connectivity," *SIAM Journal on Control and Optimization*, vol. 50, no. 5, pp. 2588–2620, Jan. 2012.
 - [30] L. Sabattini, N. Chopra, and C. Secchi, "On decentralized connectivity maintenance for mobile robotic systems," in *Proceedings of the IEEE Conference on Decision and Control and European Control Conference*, Dec. 2011, pp. 988–993.
 - [31] M. M. Zavlanos and G. J. Pappas, "Distributed connectivity control of mobile networks," *IEEE Transactions on Robotics*, vol. 24, no. 6, pp. 1416–1428, Dec. 2008.
 - [32] K. Khateri, M. Pourgholi, M. Montazeri, and L. Sabattini, "A comparison between decentralized local and global methods for connectivity maintenance of multi-robot networks," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 633–640, Apr. 2019.
 - [33] M. Zavlanos, A. Ribeiro, and G. Pappas, "Network integrity in mobile robotic networks," *IEEE Transactions on Automatic Control*, vol. 58, no. 1, pp. 3–18, 2013.
 - [34] E. I. Grøtli and T. A. Johansen, "Task assignment for cooperating UAVs under radio propagation path loss constraints," in *Proceedings of the American Control Conference (ACC)*, no. 978. IEEE, Jun. 2012, pp. 3278–3283.
 - [35] Y. Pei, M. W. Mutka, and N. Xi, "Connectivity and bandwidth-aware real-time exploration in mobile robot networks," *Wireless Communications and Mobile Computing*, vol. 13, no. 9, pp. 847–863, Jun. 2013.
 - [36] L. Navaravong, Z. Kan, J. M. Shea, and W. E. Dixon, "Formation reconfiguration for mobile robots with network connectivity constraints," *IEEE Network*, vol. 26, no. 4, pp. 18–24, 2012.
 - [37] J. Panerati, L. Gianoli, C. Pincirolì, A. Shabah, G. Nicolescu, and G. Beltrame, "From swarms to stars: Task coverage in robot swarms with connectivity constraints," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, May 2018, pp. 7674–7681.
 - [38] T. Nestmeyer, P. Robuffo Giordano, H. H. Bühlhoff, and A. Franchi, "Decentralized simultaneous multi-target exploration using a connected network of multiple robots," *Autonomous Robots*, vol. 41, no. 4, pp. 989–1011, 2017.
 - [39] N. Majcherczyk, A. Jayabalan, G. Beltrame, and C. Pincirolì, "Decentralized Connectivity-Preserving Deployment of Large-Scale Robot Swarms," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Oct. 2018, pp. 4295–4302.
 - [40] G. A. Hollinger and S. Singh, "Multirobot coordination with periodic connectivity: Theory and experiments," *IEEE Transactions on Robotics*, vol. 28, no. 4, pp. 967–973, Aug. 2012.
 - [41] D. Tateo, J. Banfi, A. Riva, F. Amigoni, A. Bonarini, and P. Milano, "Multiagent connected path planning: PSPACE-completeness and how to deal with it," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018, pp. 4735–4742.
 - [42] J. Banfi, N. Basilico, and F. Amigoni, "Multirobot reconnection on graphs: Problem, complexity, and algorithms," *IEEE Transactions on Robotics*, vol. 34, no. 5, pp. 1299–1314, Oct. 2018.
 - [43] J. Banfi, A. Q. Li, N. Basilico, I. Rekleitis, and F. Amigoni, "Multirobot online construction of communication maps," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, May 2017, pp. 2577–2583.
 - [44] E. I. Grøtli and T. A. Johansen, "Path planning for UAVs under communication constraints using SPLAT! and MILP," *Journal of Intelligent & Robotic Systems*, vol. 65, no. 1–4, pp. 265–282, Aug. 2011.
 - [45] B. L. Wellman, S. Dawson, J. de Hoog, and M. Anderson, "Using rendezvous to overcome communication limitations in multirobot exploration," in *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics*, Oct. 2011, pp. 2401–2406.
 - [46] J. Banfi, A. Quattrini Li, I. Rekleitis, F. Amigoni, and N. Basilico, "Strategies for coordinated multirobot exploration with recurrent connectivity constraints," *Autonomous Robots*, vol. 42, no. 4, pp. 875–894, Apr. 2018.
 - [47] E. I. Grøtli and T. A. Johansen, "Path- and data transmission planning for cooperating UAVs in delay tolerant network," in *Proceedings IEEE Globecom Workshops*. IEEE, Dec. 2012, pp. 1568–1573.

- [48] E. F. Flushing, L. M. Gambardella, and G. A. Di Caro, "Simultaneous task allocation, data routing, and transmission scheduling in mobile multi-robot teams," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Sep. 2017, pp. 1861–1868.
- [49] J. Banfi, N. Basilico, and F. Amigoni, "Minimizing communication latency in multirobot situation-aware patrolling," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Sep. 2015, pp. 616–622.
- [50] E. F. Flushing, M. Kudelski, L. M. Gambardella, and G. a. Di Caro, "Connectivity-aware planning of search and rescue missions," in *Proceedings of the IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, 2013.
- [51] S. S. Ponda, L. B. Johnson, A. N. Kopeikin, H. L. Choi, and J. P. How, "Distributed planning strategies to ensure network connectivity for dynamic heterogeneous teams," *IEEE Journal on Selected Areas in Communications*, vol. 30, no. 5, pp. 861–869, 2012.
- [52] D. A. Anisi, P. Ögren, and X. Hu, "Cooperative minimum time surveillance with multiple ground vehicles," *IEEE Transactions on Automatic Control*, vol. 55, no. 12, pp. 2679–2691, Dec. 2010.
- [53] P. Klein and R. Ravi, "A nearly best-possible approximation algorithm for node-weighted Steiner Trees," *Journal of Algorithms*, vol. 19, no. 1, pp. 104–115, Jul. 1995.
- [54] Z. Kan, L. Navaravong, J. M. Shea, E. L. Pasiliao, and W. E. Dixon, "Graph matching-based formation reconfiguration of networked agents with connectivity maintenance," *IEEE Transactions on Control of Network Systems*, vol. 2, no. 1, pp. 24–35, Mar. 2015.
- [55] S. Umeyama, "An eigendecomposition approach to weighted graph matching problems," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 10, no. 5, pp. 695–703, 1988.
- [56] S. M. LaValle, *Planning Algorithms*. Cambridge: Cambridge University Press, 2006.
- [57] J. O'Rourke, *Art Gallery Theorems and Algorithms*. New York, NY, USA: Oxford University Press, Inc, 1987.
- [58] C. Audet and J. E. Dennis, "Analysis of generalized pattern searches," *SIAM Journal on Optimization*, vol. 13, no. 3, pp. 889–903, Jan. 2002.
- [59] MathWorks. How pattern search polling works. Accessed: August 30, 2019. [Online]. Available: <https://www.mathworks.com/help/gads/how-pattern-search-polling-works.html>

• • •